Apache Cassandra

Borrador de introducción y arquitectura

1. Introducción

Apache Cassandra es una base de datos NoSQL de columnas anchas. La usan equipos que necesitan distribuir grandes volúmenes de datos entre varios servidores y mantener el servicio disponible cuando falla un nodo. Un ejemplo es Discord. Su equipo de ingeniería documentó que, a principios de 2022, almacenaba billones de mensajes en un clúster de 177 nodos Cassandra. El crecimiento también trajo problemas de latencia y mantenimiento que motivaron su migración posterior a ScyllaDB [1], [2].

Este informe analiza cómo el nivel de consistencia afecta la posibilidad de leer un dato cuando se pierden réplicas. La prueba utiliza tres nodos Cassandra en Docker y compara la misma lectura con `ONE` y `QUORUM` antes, durante y después de detener dos nodos. Como contexto, se toman estudios que comparan configuraciones de consistencia de Cassandra, MongoDB y Redis en la nube [3], [4]. La comparación arquitectónica con MongoDB se apoya en su documentación sobre conjuntos de réplicas [5]. La práctica local se limita a Cassandra y observa si la consulta puede completarse.

2. Arquitectura de Cassandra

Cassandra funciona como un clúster de nodos pares. El nodo que recibe una solicitud del cliente actúa como coordinador de esa operación y se comunica con las réplicas que corresponden al dato. Los nodos mantienen información sobre la membresía y el estado del clúster mediante el protocolo *gossip* [1].

Para ubicar los datos, Cassandra calcula un hash de la clave de partición y lo usa como token para determinar qué nodos almacenan la partición. En la tabla `prueba`, la clave de partición es `id`. El espacio de claves `ta_ind_01` usa `NetworkTopologyStrategy` con un factor de replicación de tres en `datacenter1`. Así, los tres nodos guardan una copia de la partición consultada. El nivel de consistencia indica cuántas réplicas deben responder para completar una operación. En esta configuración, una lectura con `ONE` requiere una respuesta y con `QUORUM` requiere dos [1].

La Figura 1 muestra el recorrido de la consulta en el montaje de Cassandra 5.0.9. Desde PowerShell se ejecuta `cqlsh` dentro del contenedor del nodo 1, mediante `docker exec`. El nodo 1 coordina la consulta y también almacena una réplica. Los nodos 2 y 3 guardan las otras dos. Los tres contenedores se ejecutan en una sola computadora, por lo que el montaje permite observar fallos de nodos individuales dentro del mismo equipo.

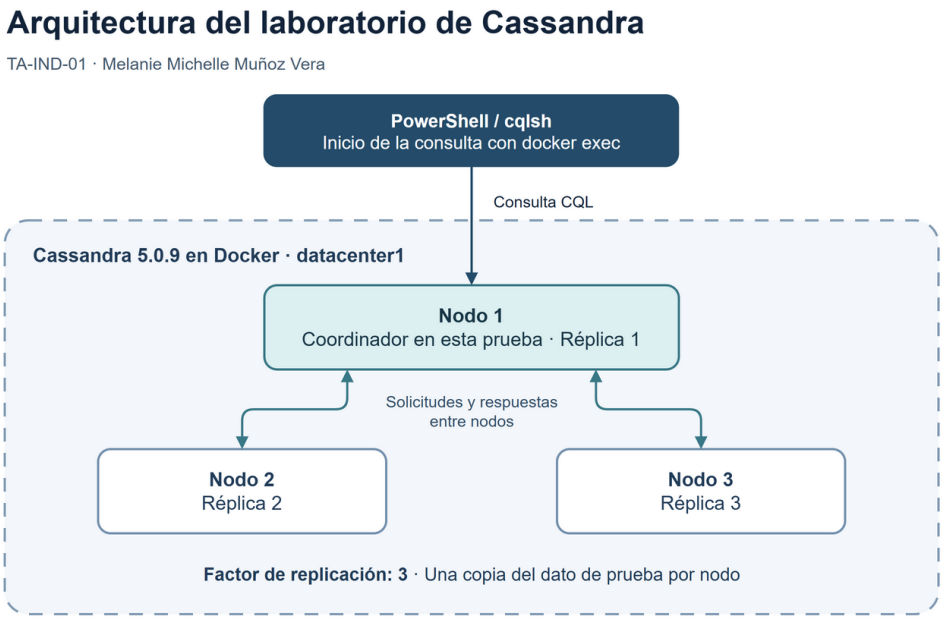


Figura 1: Recorrido de una consulta CQL en el laboratorio. Diagrama elaborado a partir del bosquejo del Anexo A.

Referencias

[1] Apache Cassandra Project. «Dynamo.» Apache Cassandra Documentation, versión 5.0, visitado 12 de nov. de 2026. dirección: `https://cassandra.apache.org/doc/stable/cassandra/architecture/dynamo.html`

[2] B. Ingram. «How Discord Stores Trillions of Messages,» Discord, visitado 13 de nov. de 2026. dirección: `https://discord.com/blog/how-discord-stores-trillions-of-messages`

[3] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo y E. Andrade, «Benchmarking Consistency Levels of Cloud-Distributed NoSQL Databases Using YCSB,» *IEEE Access*, vol. 13, págs. 63 428-63 438, 2025, doi: `10.1109/ACCESS.2025.3558923`.

[4] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo y E. Andrade, «Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications,» *Journal of Cloud Computing*, vol. 13, n.º 1, 2024, art. 158, doi: `10.1186/s13677-024-00716-7`.

[5] MongoDB. «Replication.» MongoDB Database Manual, versión 8.0, visitado 12 de nov. de 2026. dirección: `https://www.mongodb.com/docs/v8.0/replication/`